Stellar Cyber 6.6.0s is now live! Discover all the updates in the release notes!

Installing a Modular Sensor in KVM (SaaS)

This topic describes how to install a Modular Sensor on a KVM virtual machine using the installation script and QCOW2 image file downloaded from the **Download Images** tab in the **System | DATA SOURCE MANAGEMENT | Sensors | Sensor Installation** page. Refer to the following sections for details:

- **About Modular Sensors**
- **Prerequisites**
- **Obtaining the Installation File**
- **Installing in Ubuntu 20.04/22.04/24.04**
- **Installing in Ubuntu Server 16.04, 18.04, or CentOS 7.3**
- **Applying a Token to the Installed Sensor**
- **Configuring a Static IP Address (Optional)**
- **Configure NTP and Set the Timezone**
- **Enabling SSSE3 for the Modular Sensor VM**

Use our example as a guideline, as you might be using a different software version.

Stellar Cyber does not support the installation of third-party software on its virtual or physical device sensors.

About Modular Sensors

Sensors provide the data gathering foundation for Stellar Cyber's OpenXDR platform, gathering the right data with context. Modular sensors are purpose-built Stellar Cyber sensors that include both the host and the Stellar Cyber monitoring software. They are provided as both physical devices (Photon sensors) and virtual machine images for different target environments.

Previous releases provided a variety of different types of device sensors, including Network, Security, and Modular. Going forward, the only type of device sensor is Modular. You can

use the Modular Sensor Profile to enable whatever sensor features you like, creating the same functionality provided by the different sensor types in previous releases.

A modular sensor lets you easily add the features you like to your sensor. This helps simplify your deployment and lets you manage the VM requirements for the sensors based on the modular features they use.

Modular Sensors always include log ingestion. From there, you can enable different features as part of your modular sensor profile:

- Enable the **Network Traffic** feature to monitor the virtual environment, the physical environment if connected to the span port of a physical switch, or the LAN segment via a mirror port on a switch. The sensor monitors network and server response times and can identify applications.

  The sensor converts that information to metadata and forwards it to the DP as Interflow. The DP can then provide security, DDoS, and breach attempt detections.
- Enable the **Sandbox** and **IDS** features to improve your security posture:
  - Sandbox lets you detect malware in files and network traffic through Stellar Cyber's integrated cloud service and also provides anti-virus services.
  - IDS lets you detect intrusion attempts using both files and network traffic.

Keep in mind that VM resource requirements increase as you add more features to the Modular Sensor Profile. Refer to **Virtual Modular Sensor Provisioning and Performance** for details on the resources required to run different combinations of features in a Modular Sensor Profile, as well as how to use the **show module** and **show module request** CLI commands to compare provisioned resources against those required to run specific feature combinations. Stellar Cyber only enables a Modular Sensor Profile on a sensor if the host VM's resources can support it.

Prerequisites

You can install a KVM modular sensor in the following environments:

- CentOS 7.3 (or later)
- Red Hat Enterprise Linux 9.x
- Ubuntu Server 16.04, 18.04, 20.04, 22.04, or 24.04

Refer to **Virtual Modular Sensor Provisioning and Performance** for details on the resources required to run different combinations of features in a Modular Sensor Profile. Provision your modular sensor according to the features that you plan on enabling.

You will need:

- DPDK-capable Ethernet port(s) are recommended.
- One IP address with access to a default gateway.
- A token obtained from the DP that will manage the sensor.
- Open firewall ports for log ingestion.

- Open firewall ports for the features you plan on enabling in the Modular Sensor Profile for this sensor.

Verify VM Capabilities

Before installing any software, verify whether the system has the VM capabilities required. This can be done from the command line using the instructions below for Intel-based and AMD-based system:

Intel-Bases Systems

For Intel-based systems use the command:

```
cat /proc/cpuinfo | grep vmx
```

If no lines are listed then VM hardware support is not available. It must be enabled in the system BIOS.

AMD-Based Systems

For AMD-based systems use:

```
cat /proc/cpuinfo | grep svm
```

One line will be listed for each secure-VM core available. If no lines are listed then VM hardware support must be enabled in the system BIOS.

If VM capability is not reported by these commands, do not proceed until it is enabled.

Installing KVM and Linux Bridge Tools

The system requires KVM and Linux bridge tools. Install these as follows, depending on your version:

Red Hat Enterprise Linux 9.x

```
sudo dnf install -y net-tools qemu-kvm qemu-img virt-manager libvirt python3-libvirt
libvirt-client virt-install virt-viewer
sudo dnf install -y https://dl.fedoraproject.org/pub/epel/epel-release-latest-
9.noarch.rpm
sudo dnf install -y bridge-utils
```

Ubuntu 16.04/18.04

```
sudo apt-get update
sudo apt-get install -y qemu-kvm libvirt-bin virtinst bridge-utils qemu-utils
virtinst virt-viewer genisoimage net-tools cpu-checker
```

Ubuntu 20.04/22.04/24.04

```
sudo apt-get update
sudo apt-get install -y qemu-kvm libvirt-daemon-system libvirt-clients virtinst
bridge-utils qemu-utils virt-viewer genisoimage net-tools cpu-checker
```

CentOS

```
yum install net-tools qemu-kvm qemu-img virt-manager libvirt libvirt-python libvirt-
client virt-install virt-viewer bridge-utils
```

```
systemctl start libvirtd
systemctl enable libvirtd
```

Verify libvirt Daemon is Active

Use one of the following commands to verify that the libvirt daemon is active:

- `systemctl is-active libvirtd`
- `systemctl status libvirtd`

If libvirtd is not active, start it now with the following command:

```
sudo systemctl enable --now libvirtd
```

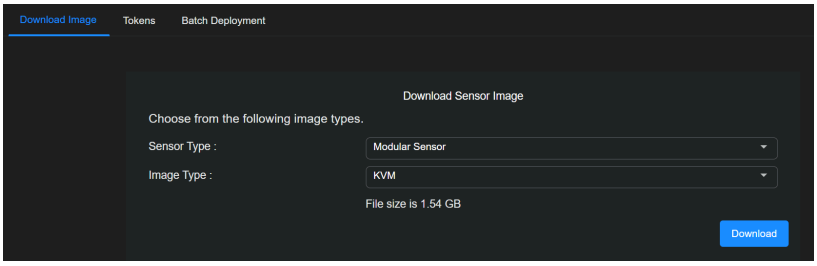
Obtaining the Installation File

You download the installation file for the Modular Sensor for KVM from the **Download Images** tab in the **System | DATA SOURCE MANAGEMENT | Sensors | Sensor Installation page**. Use the following procedure:

Only users with the **System | DATA SOURCE MANAGEMENT | Sensors | Sensor Installation | Sensor Image Download** privilege assigned to their profile in the System | ORGANIZATION MANAGEMENT | Role-Based Access Privileges interface can download images.

1. Navigate to the **System | DATA SOURCE MANAGEMENT | Sensors | Sensor Installation** page.
2. Set the **Sensor Type** dropdown to **Modular Security Sensor**.
3. Set the **Image Type** to **KVM**.

   The display updates to show you the size of the file to be downloaded.



4. Click the **Download** button. The system downloads the installation file (**aella-modular-ds-6.x.x.qcow2.zip**) along with its corresponding SHA-1 hash file.
5. Unzip the installation file's contents (**virt_deploy_modular_ds.sh** and **aella-modular-ds-6.x.x.qcow2**).

Installing in Red Hat Enterprise Linux 9.x 6.2.0

Click here to see the installation instructions for Red Hat Enterprise Linux 9.x)

This section describes how to install a modular sensor in a Red Hat Enterprise Linux environment.

Check the Host Resources and Interfaces

Before you start installing the sensor, you should check the configuration of the host server's network interfaces as well as its available system resources. Use the following commands to check the current system resources:

- `lscpu` – Reports the total number of CPUs.
- `free -h` – Reports the total system memory.
- `df -h` – Reports the total system disk size.

You will need to take all these values under consideration when provisioning resources for the sensor VM later on.

Configure the Linux Bridge

Use the procedure below to create and configure a network bridge using NetworkManager's command-line tool (**nmcli**).

1. Remove the existing network configuration for the physical network interface **eno1**. This ensures that no prior connection settings conflict with the new bridge setup.

   ```
   # sudo nmcli connection delete eno1
   ```

2. Create a new bridge interface named **br0-aio**:

   ```
   # sudo nmcli connection add type bridge ifname br0-aio con-name br0-aio
   ```

   This command creates a virtual bridge device that can manage traffic between physical and virtual interfaces on the same Layer 2 network.

3. Assign a static IPv4 address, gateway, and DNS servers to the bridge with the following commands in sequence, substituting your own values for the sample values:

   ```
   sudo nmcli connection modify br0-aio ipv4.addresses 10.16.11.101/22
   sudo nmcli connection modify br0-aio ipv4.gateway 10.16.8.1
   sudo nmcli connection modify br0-aio ipv4.dns "10.16.7.8 208.91.112.52"
   sudo nmcli connection modify br0-aio ipv4.method manual
   sudo nmcli connection modify br0-aio ipv6.method ignore
   ```

4. Associate the physical Ethernet interface, **eno1**, with the bridge. This lets the bridge use **eno1** for external network communication.

   ```
   # sudo nmcli connection add type ethernet con-name eno1-br0 ifname eno1 master br0-aio
   ```

   This command creates a new Ethernet connection named **eno1-br0** and assigns it as a slave to the **br0-aio** bridge.

5. Bring both the bridge and the slave interface online to apply the configuration.

   ```
   # sudo nmcli connection up br0-aio
   ```

   ```
   # sudo nmcli connection up eno1-br0
   ```

   These commands activate the connections immediately without requiring a reboot.

6. Run the following command to confirm that the bridge and associated interfaces are configured correctly:

```
# nmcli connection show
```

A successful configuration displays output similar to the following:

```
NAME      UUID                                  TYPE      DEVICE
br0-aio   67736b98-98b4-416b-b753-2f377d774274  bridge    br0-aio
eno1-br0  0e5f8be2-27ba-4a80-935f-34ed6d3f6149  ethernet  eno1
lo        88211344-6d78-4213-a393-9b5a0410ff96  loopback  lo
eno2      0886abe7-ee20-4219-b978-7cec00eb9643  ethernet  --
ens15f0   48909cc0-4041-44a3-b4cf-136716dcf808  ethernet  --
ens15f1   b95d4c85-0a66-4def-8998-f07b6b4bc94e  ethernet  --
```

Summary:

After completing these steps:

- The bridge interface **br0-aio** is active and configured with static IP settings.
- The physical interface **eno1** is successfully attached as a slave to the bridge.
- The system is now ready for use in a virtualized KVM host environment.

Verify KVM Support

1. Run the following command to verify that the KVM modules are loaded and that the KVM device file is present and accessible. This check ensures that virtualization is properly supported and enabled on the host system:

   ```
   # lsmod | grep kvm
   ```

2. In response, you should see text similar to the following:

   ```
   kvm_intel             446464  0
   kvm                  1404928  1 kvm_intel
   ```

   - The presence of **kvm_intel** (or **kvm_amd**, depending on CPU type) indicates that the CPU-specific KVM module is loaded.
   - The **kvm** line confirms that the core KVM module is active.

3. Verify that the KVM device file exists and has the correct permissions with the following command:

   ```
   # ls -l /dev/kvm
   ```

4. In response, you should see output similar to the following:

   ```
   crw-rw-rw-. 1 root kvm 10, 232 Sep 8 01:30 /dev/kvm
   ```

   - The leading **c** indicates a character device file.
   - Permissions (**rw-rw-rw-**) mean the device is readable and writable by all users.
   - The owner is **root**, and the group is **kvm**.

5. Validate KVM host virtualization capabilities with the following command:

   ```
   # virt-host-validate
   ```

   This command checks for all essential components required for KVM/QEMU-based virtualization. Make sure the command output is similar to the following:

```
# virt-host-validate
  QEMU: Checking for hardware
virtualization                                 : PASS
  QEMU: Checking device '/dev/kvm'
exists                                         : PASS
  QEMU: Checking if device '/dev/kvm' is
accessible                                     : PASS
  QEMU: Checking if device '/dev/vhost-net'
exists                                         : PASS
  QEMU: Checking if device '/dev/net/tun'
exists                                         : PASS
  QEMU: Checking for cgroup 'cpu' controller
support                                        : PASS
  QEMU: Checking for cgroup 'cpuacct' controller
support                                        : PASS
  QEMU: Checking for cgroup 'cpuset' controller
support                                        : PASS
  QEMU: Checking for cgroup 'memory' controller
support                                        : PASS
  QEMU: Checking for cgroup 'devices' controller
support                                        : PASS
  QEMU: Checking for cgroup 'blkio' controller
support                                        : PASS
  QEMU: Checking for device assignment IOMMU
support                                        : PASS
  QEMU: Checking if IOMMU is enabled by
kernel                                         : WARN (IOMMU appears to be disabled
in kernel. Add intel_iommu=on to kernel cmdline arguments)
  QEMU: Checking for secure guest
support                                        : WARN (Unknown if this
platform has Secure Guest support)
#
```

Create Installation Directory for Virtual Machine and Installation Script

Use the following commands to create an installation directory for your virtual machine and unzip the contents of the zip archive you downloaded in **Obtaining the Installation File** to it:

Default Directory	`sudo mkdir -p /var/lib/libvirt/images/mds/images` `sudo mv aella-modular-ds-6.6.0.qcow2* /var/lib/libvirt/images/mds/images`
Non-Default Directory	`sudo mkdir -p /stellar/mds/images` `sudo mv aella-modular-ds-6.6.0.qcow2* /stellar/mds/images/`

Install the Sensor

This section describes how to install the sensor using the image and installation script you previously downloaded. Note the following regarding the example in this section:

- Do not include the **--span** argument with the `virt_deploy_modular_ds.sh` installation script when installing in Red Hat Enterprise Linux.
- The example in this section assumes that the image has already been downloaded (the **nodownload** parameter is set to **true**). You can also set **nodownload** to **false** and have the script pull the image as part of the installation.
- You can include installation parameters directly in the installation script's command line OR you can declare them as variables and then pass them to the script in the command line. This has the advantage of reusability and is the example provided below

1. Switch to the root user with the `sudo su` command.

2. Set the following variables, making sure to change the installdir to match the directory where you stored the image in the previous section. Note that this example creates a VM with 16 vCPUs, 32GB of memory, and a 128GB disk size. You can use the guidelines in **Server Sensor Sizing Specifications** to determine the VM size you need for your anticipated needs.

   The installation example at the end of this procedure provides the exact syntax to declare and pass these variables.

Variable	Comments
version=6.6.0s	
hostname=mds	Set as desired.
release=6.6.0s	
installdir=/var/lib/libvirt/images/mds	Set to match the directory where you copied the sensor image in the previous section.
cpus=16	
memory=$(expr 32 * 1024)	32GB
disksize=128 # VM disk size (e.g. DO NOT put G suffix)	
nodownload=true	When set to true, the installation script does not download the sensor image but uses the one in the specified `installdir`.
bridge=br0-aio	Management bridge name.
ip=10.33.2.98	VM IP address.
netmask=255.255.255.0	VM netmask.
gw=10.33.2.1	VM default gateway.
dns=8.8.8.8	VM DNS server address.

3. Use the following command to verify that the installation parameters are set correctly based on the table in the previous step:

```
printf "\n bash virt_deploy_modular_ds.sh -- --hostname=$hostname --release=$release --CPUS=$cpus --MEM=$memory --DISKSIZE=$disksize --installdir=$installdir --nodownload=$nodownload --bridge=$bridge --ip=$ip --netmask=$netmask --gw=$gw --dns=$dns \n"
```

4. Use the following command to start the deployment:

```
bash virt_deploy_modular_ds.sh -- --hostname=$hostname --release=$release --CPUS=$cpus --MEM=$memory --DISKSIZE=$disksize --installdir=$installdir --nodownload=$nodownload --bridge=$bridge --ip=$ip --netmask=$netmask --gw=$gw --dns=$dns
```

If you already installed the libvirt-daemon-system daemon, you can ignore any errors similar to: Package 'libvirt-bin' has no installation candidate".

5. Verify the installation with the `virsh list --all` command. You should see the newly created MDS VM listed in the output.

Sample Syntax

Click here to see the complete set of commands in the installation procedure:

```
# version=6.6.0
# hostname=mds
# release=$version
# installdir=/stellar/mds
# cpus=16
# memory=$(expr 32 \* 1024) # 32GB
# disksize=128              # VM disk size (e.g. DO NOT put G suffix)
# nodownload=true           # skip to download the modular sensor image
#
# bridge=br0-aio            # management bridge name
#
# ip=10.33.2.98             # VM IP address
# netmask=255.255.255.0     # VM IP netmask
# gw=10.33.2.1              # VM IP gateway
# dns=8.8.8.8               # VM DNS server address
# printf "\n bash virt_deploy_modular_ds.sh -- --hostname=$hostname --
release=$release --CPUS=$cpus --MEM=$memory --DISKSIZE=$disksize  --
installdir=$installdir --nodownload=$nodownload --bridge=$bridge --ip=$ip --
netmask=$netmask --gw=$gw --dns=$dns \n"

 bash virt_deploy_modular_ds.sh -- --hostname=mds --release=6.6.0 --CPUS=8 --
MEM=32768 --DISKSIZE=128  --installdir=/stellar/mds --nodownload=true --
bridge=br0-aio --ip=10.33.2.98 --netmask=255.255.255.0 --gw=10.33.2.1 --
dns=8.8.8.8

# bash virt_deploy_modular_ds.sh -- --hostname=$hostname --release=$release --
CPUS=$cpus --MEM=$memory --DISKSIZE=$disksize --installdir=$installdir --
nodownload=$nodownload --bridge=$bridge --ip=$ip --netmask=$netmask --gw=$gw --
dns=$dns
Changing memory to user customized memory  + 32768
Changing cpu to user customized cpu size  + 16
Wed, 17 Aug 2025 00:52:31 +0000 Destroying the mds domain (if it exists)...
/stellar/mds/images/aella-modular-ds-6.6.0.qcow2: OK
Checksum for /stellar/mds/images/aella-modular-ds-6.6.0.qcow2 success
Wed, 17 Aug 2025 00:52:34 +0000 Converting Qcow2 image to RAW format...
WARNING: Image format was not specified for 'mds.raw' and probing guessed raw.
         Automatically detecting the format is dangerous for raw images, write
operations on block 0 will be restricted.
         Specify the 'raw' format explicitly to remove the restrictions.
Image resized.
image: mds.raw
file format: raw
virtual size: 128 GiB (137438953472 bytes)
disk size: 2.22 GiB
Wed, 17 Aug 2025 00:52:38 +0000 Installing the domain and adjusting the
configuration...

Starting install...
Domain creation completed.
checking host 10.33.2.98
Fail to ping host 10.33.2.98
Waiting for ssh ready.
.ssh - ok
\n
Domain mds defined from domain-xml

Gen uninstall scripts...
Check Host health again after restarting...
checking host 10.33.2.98
ping host - ok
Waiting for ssh ready.
ssh - ok
\n
Wed, 17 Aug 2025 00:53:34 +0000 SSH to mds using 10.33.2.98 with  username
'aella'.
```

Attach a virtual interface to Modular Sensor VM and connect it to the traffic monitoring bridge (br0-span)

1. Shut down the MDS VM with the following command:

   ```
   virsh shutdown mds
   ```

   Substitute the name of your VM for **mds**.

2. Next, you need to attach a virtual interface to the Modular Sensor VM and connect it to the traffic monitoring bridge (**br0-span** in our example).Use the following command:

   ```
   virsh attach-interface --domain $hostname --type bridge --source $spanbr --
   model virtio --config --live --persistent
   ```

   
   Set the **$hostname** to the name you specified for your sensor VM during deployment (**mds** in this example).

3. Use the following command to verify that you can see the attached interface in the Modular Sensor VM. Substitute the name of your host if you did not use mds:

   ```
   # virsh domiflist mds
   Interface Type Source Model MAC
   -------------------------------------------------------------
   vnet0 bridge br0-aio virtio 52:54:00:f3:3a:63
   vnet1 bridge br0-span virtio 52:54:00:38:11:80 <<< LOOK HERE
   ```

Next Steps

Next, you'll obtain and apply the token used to authorize and configure the installed sensor. Proceed to **Applying a Token to the Installed Sensor**.

Installing in Ubuntu 20.04/22.04/24.04

Click here to see the installation instructions for Ubuntu 20.04 or 22.04:

This section describes how to install a modular sensor in an Ubuntu 20.04/22.04/24.04 environment. Note the following:

- The major difference between the instructions for Ubuntu 20+ and those for Ubuntu 16.04/18.04 is that Ubuntu 20.04+ uses the `netplan` utility for its network configuration.
- The netplan configuration format changed in Ubuntu 22.04+. Separate examples of the relevant configuration files are provided for Ubuntu 20.04 and Ubuntu 22.04+ so you can see the differences.

Check the Host Resources and Interfaces

Before you start installing the sensor, you should check the configuration of the host Ubuntu server's network interfaces as well as its available system resources. Use the following commands to check the current system resources:

- `lscpu` – Reports the total number of CPUs.

- `free -h` – Reports the total system memory.
- `df -h` – Reports the total system disk size.

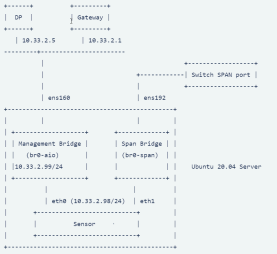
You will need to take all these values under consideration when provisioning resources for the sensor VM later on.

Sample Host Configuration

For our sample installation, the host Ubuntu server has two physical network interfaces (**ens160** and **ens192**) and will be configured as follows:

- **ens160** – For management connection
- **ens192** – For monitoring traffic mirrored from a switch SPAN port

The figure below summarizes the sample host environment:



Create Bridges for Management and Traffic Monitoring

Use the following procedure to create bridges for management and traffic monitoring:

1. Check the current management interface configuration by viewing the contents of the YAML files stored in **/etc/netplan**. For example, here's how the **00-installer-config.yaml** file for the **ens160** management interface would look for both static IP and DHCP configurations in different Ubuntu versions (note that the netplan configuration format changed in Ubuntu 22.04, replacing `gateway4` with `to` and `via` routes:

    - **Static IP (Ubuntu 20.04)**

    ```
    network:
      ethernets:
        ens160:
          addresses:
          - 10.33.2.99/24
          gateway4: 10.33.2.1
          nameservers:
            addresses:
            - 8.8.8.8
            search: []
      version: 2
    ```

    - **Static IP (Ubuntu 22.04 and Later)**

    ```
    network:
      ethernets:
        ens160:
          dhcp4: no
          dhcp6: no
          addresses:
            - 10.33.2.99/24
          route:
            - to: default
              via: 10.33.2.1
          nameservers:
            addresses: [8.8.8.8, 8.8.4.4]
    ```

```
        search: []
  version: 2
```

- **DHCP (Ubuntu 20.04)**

```
network:
  ethernets:
    ens160:
      dhcp4: true
  version: 2
```

- **DHCP (Ubuntu 22.04 and Later))**

```
network:
  ethernets:
    ens160:
      dhcp4: yes
      dhcp6: no
  version: 2
```

2. Make a backup copy of the network configuration file. For example, for the **00-installer-config.yaml** file:

   ```
   cd /etc/netplan
   sudo cp 00-installer-config.yaml 00-installer-config.yaml.orig
   ```

3. Create bridges for the management interface and traffic mirroring interface by editing the **00-installer-config.yaml** file. For our example:

   - The management interface (**ens160**) is configured to the **br0-aio** bridge.
   - The traffic mirroring interface (**ens192**) is configured to the **br0-span** bridge.

   Separate examples are provided below for both static IP and DHCP configurations in both Ubuntu 20.04 and Ubuntu 22.04 and later:

   ### Static IP Address (Ubuntu 20.04)

   Click here to see how **00-installer-config.yaml** looks for an Ubuntu 20.04 server using a static IP address. In this case the management IP address is configured on the **br0-aio** bridge instead of **ens160**:

```
network:
  ethernets:
    ens160:
      dhcp4: false
      dhcp6: false
    ens192:
      dhcp4: false
      dhcp6: false
  bridges:
    br0-aio:
      interfaces: [ ens160 ]
      addresses:
      - 10.33.2.99/24
      gateway4: 10.33.2.1
      nameservers:
        addresses:
        - 8.8.8.8
        search: []
      parameters:
        stp: false
        forward-delay: 0
    br0-span:
      interfaces: [ ens192 ]
      parameters:
        stp: false
        forward-delay: 0
```

```
  version: 2
```

Static IP Address (Ubuntu 22.04 and Later))

Click here to see how **00-installer-config.yaml** looks for an Ubuntu 22.04+ server using a static IP address. In this case the management IP address is configured on the **br0-aio** bridge instead of **ens160**:

```
network:
  ethernets:
    ens160:
      dhcp4: no
      dhcp6: no
    ens192:
      dhcp4: no
      dhcp6: no
  bridges:
    br0-aio:
      interfaces: [ ens160 ]
      addresses:
      - 10.33.2.99/24
      route:
      - to: default
          via: 10.33.2.1
      nameservers:
        addresses:
        - [8.8.8.8, 8.8.4.4]
        search: []
      parameters:
        stp: false
        forward-delay: 0
    br0-span:
      interfaces: [ ens192 ]
      parameters:
        stp: false
        forward-delay: 0

  version: 2
```

DHCP Address (Ubuntu 20.04)

Click here to see how **00-installer-config.yaml** looks for an Ubuntu 20.04 server using DHCP. In this case, DHCP is enabled on the **br0-aio** bridge instead of **ens160**:

```
network:
  ethernets:
    ens160:
      dhcp4: false
      dhcp6: false
    ens192:
      dhcp4: false
      dhcp6: false
  bridges:
    br0-aio:
      interfaces: [ ens160 ]
      dhcp4: true
      dhcp6: false
      parameters:
        stp: false
        forward-delay: 0
    br0-span:
      interfaces: [ ens192 ]
      dhcp4: false
      dhcp6: false
      parameters:
        stp: false
        forward-delay: 0

  version: 2
```

DHCP Address (Ubuntu 22.04 and Later)

Click here to see how **00-installer-config.yaml** looks for an Ubuntu 22.04+ server using DHCP. In this case, DHCP is enabled on the **br0-aio** bridge instead of **ens160**:

```
network:
  ethernets:
    ens160:
      dhcp4: no
      dhcp6: no
    ens192:
      dhcp4: no
      dhcp6: no
  bridges:
    br0-aio:
      interfaces: [ ens160 ]
      dhcp4: yes
      dhcp6: no
      parameters:
        stp: false
        forward-delay: 0
    br0-span:
      interfaces: [ ens192 ]
      dhcp4: no
      dhcp6: no
      parameters:
        stp: false
        forward-delay: 0

  version: 2
```

4. Apply your changes to the netplan configuration with the following command:

   ```
   sudo netplan apply
   ```

5. Use the **brctl** show command to make sure you can see both the **br0-aio** and **br0-span** bridges. For example:

   ```
   root@ubuntu-20-04-kvm:$ brctl show
   bridge name bridge id STP enabled interfaces
   br0-aio 8000.000c29226a14 no ens160 <<< look here
   br0-span 8000.000c29226a1e no ens192 <<<look here
   virbr0 8000.525400221195 yes virbr0-nic
   ```

6. Use the **ip addr** command to verify that the management IP address is assigned to **br0-aio**.

   Click here to see an example:

```
aella@ubuntu-20-04-kvm:~$ ip addr
                                                ...
                                                2: ens160:
<BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq master br0-mgt state
UP group default qlen 1000
                                                link/ether 00:0c:29:22:6a:14 brd
ff:ff:ff:ff:ff:ff
                                                ...
                                                5: br0-aio:
<BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group
default qlen 1000
                                                link/ether 00:0c:29:22:6a:14 brd
ff:ff:ff:ff:ff:ff
                                                inet 10.33.2.99/24 brd
10.33.2.255 scope global br0-mgt
                                                valid_lft forever preferred_lft
forever
                                                inet6
fe80::402e:31ff:fe54:f037/64 scope link
                                                valid_lft forever preferred_lft
forever
                                                6: ens192:
<BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP group
default qlen 1000
                                                link/ether 00:0c:29:22:6a:1e brd
ff:ff:ff:ff:ff:ff
                                                inet6
```

```
fe80::20c:29ff:fe22:6a1e/64 scope link
                                        valid_lft forever preferred_lft
forever
                                        ...
                                        9: br0-span:
<BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group
default qlen 1000
                                        link/ether 00:0c:29:22:6a:1e brd
ff:ff:ff:ff:ff:ff
                                        inet6
fe80::88a9:c7ff:feb1:9fc/64 scope link
                                      valid_lft forever preferred_lft
forever
```

Create Installation Directory for Virtual Machine

Use the following commands to create an installation directory for your virtual machine and move the image you downloaded in **Obtaining the Installation File** to it:

Default Directory	`sudo mkdir -p /var/lib/libvirt/images/mds/images` `sudo mv aella-modular-ds-6.6.0.qcow2* /var/lib/libvirt/images/mds/images`
Non-Default Directory	`sudo mkdir -p /stellar/mds/images` `sudo mv aella-modular-ds-6.6.0.qcow2* /stellar/mds/images/`

Install the Sensor

This section describes how to install the sensor using the image and installation script you previously downloaded. Note the following regarding the example in this section:

- Do not include the **--span** argument with the `virt_deploy_modular_ds.sh` installation script when installing in Ubuntu 20.04/22.04/24.04. The **--span** argument is only used for installations in Ubuntu 16.04, Ubuntu 18.04, and CentOS 7.3.
- The example in this section assumes that the image has already been downloaded (the **nodownload** parameter is set to **true**). You can also set **nodownload** to **false** and have the script pull the image as part of the installation.
- You can include installation parameters directly in the installation script's command line OR you can declare them as variables and then pass them to the script in the command line. This has the advantage of reusability and is the example provided below

1. Switch to the root user with the `sudo su` command.
2. Set the following variables, making sure to change the installdir to match the directory where you stored the image in the previous section.Note that this example creates a VM with 16 vCPUs, 32GB of memory, and a 128GB disk size. You can use the guidelines in **Server Sensor Sizing Specifications** to determine the VM size you need for your anticipated needs.

   The installation example at the end of this procedure provides the exact syntax to declare and pass these variables.

Variable	Comments
version=6.6.0	
hostname=mds	Set as desired.
release=6.6.0s	
installdir=/var/lib/libvirt/images/mds	Set to match the directory where you copied the sensor image in the previous section.
cpus=16	
memory=$(expr 32 * 1024)	32GB
disksize=128 # VM disk size (e.g. DO NOT put G suffix)	
nodownload=true	When set to true, the installation script does not download the sensor image but uses the one in the specified `installdir`.
bridge=br0-aio	Management bridge name.
ip=10.33.2.98	VM IP address.
netmask=255.255.255.0	VM netmask.
gw=10.33.2.1	VM default gateway.
dns=8.8.8.8	VM DNS server address.

3. Use the following command to verify that the installation parameters are set correctly based on the table in the previous step:

```
printf "\n bash virt_deploy_modular_ds.sh -- --hostname=$hostname --release=$release --CPUS=$cpus --MEM=$memory --DISKSIZE=$disksize --installdir=$installdir --nodownload=$nodownload --bridge=$bridge --ip=$ip --netmask=$netmask --gw=$gw --dns=$dns \n"
```

4. Use the following command to start the deployment:

```
bash virt_deploy_modular_ds.sh -- --hostname=$hostname --release=$release --CPUS=$cpus --MEM=$memory --DISKSIZE=$disksize --installdir=$installdir --nodownload=$nodownload --bridge=$bridge --ip=$ip --netmask=$netmask --gw=$gw --dns=$dns
```



If you already installed the libvirt-daemon-system daemon, you can ignore any errors similar to, "E: Package 'libvirt-bin' has no installation candidate".

Sample Syntax

Click below to see the complete set of commands in the installation procedure:

```
aella@ubuntu-20-04-kvm:~# version=6.6.0
aella@ubuntu-20-04-kvm:~# hostname=mds
aella@ubuntu-20-04-kvm:~# release=$version
```

```
aella@ubuntu-20-04-kvm:~# installdir=/stellar/mds
aella@ubuntu-20-04-kvm:~# cpus=16
aella@ubuntu-20-04-kvm:~# memory=$(expr 32 \* 1024) # 32GB
aella@ubuntu-20-04-kvm:~# disksize=128              # VM disk size (e.g. DO NOT
put G suffix)
aella@ubuntu-20-04-kvm:~# nodownload=true           # skip to download the
modular sensor image
aella@ubuntu-20-04-kvm:~#
aella@ubuntu-20-04-kvm:~# bridge=br0-aio            # management bridge name
aella@ubuntu-20-04-kvm:~#
aella@ubuntu-20-04-kvm:~# ip=10.33.2.98             # VM IP address
aella@ubuntu-20-04-kvm:~# netmask=255.255.255.0     # VM IP netmask
aella@ubuntu-20-04-kvm:~# gw=10.33.2.1              # VM IP gateway
aella@ubuntu-20-04-kvm:~# dns=8.8.8.8               # VM DNS server address
aella@ubuntu-20-04-kvm:~# printf "\n bash virt_deploy_modular_ds.sh -- --
hostname=$hostname --release=$release --CPUS=$cpus --MEM=$memory --
DISKSIZE=$disksize  --installdir=$installdir --nodownload=$nodownload --
bridge=$bridge --ip=$ip --netmask=$netmask --gw=$gw --dns=$dns \n"

bash virt_deploy_modular_ds.sh -- --hostname=mds --release=6.6.0 --CPUS=8 --
MEM=32768 --DISKSIZE=128  --installdir=/stellar/mds --nodownload=true --
bridge=br0-aio --ip=10.33.2.98 --netmask=255.255.255.0 --gw=10.33.2.1 --
dns=8.8.8.8

aella@ubuntu-20-04-kvm:~# bash virt_deploy_modular_ds.sh -- --hostname=$hostname
--release=$release --CPUS=$cpus --MEM=$memory --DISKSIZE=$disksize --
installdir=$installdir --nodownload=$nodownload --bridge=$bridge --ip=$ip --
netmask=$netmask --gw=$gw --dns=$dns
Changing memory to user customized memory  + 32768
Changing cpu to user customized cpu size  + 16
Wed, 17 Aug 2022 00:52:31 +0000 Destroying the mds domain (if it exists)...
/stellar/mds/images/aella-modular-ds-6.6.0.qcow2: OK
Checksum for /stellar/mds/images/aella-modular-ds-6.6.0.qcow2 success
Wed, 17 Aug 2022 00:52:34 +0000 Converting Qcow2 image to RAW format...
WARNING: Image format was not specified for 'mds.raw' and probing guessed raw.
Automatically detecting the format is dangerous for raw images, write operations
on block 0 will be restricted.
Specify the 'raw' format explicitly to remove the restrictions.
Image resized.
image: mds.raw
file format: raw
virtual size: 128 GiB (137438953472 bytes)
disk size: 2.22 GiB
Wed, 17 Aug 2022 00:52:38 +0000 Installing the domain and adjusting the
configuration...

Starting install...
Domain creation completed.
checking host 10.33.2.98
Fail to ping host 10.33.2.98
Waiting for ssh ready.
.ssh - ok
\n
Domain mds defined from domain-xml

Gen uninstall scripts...
Check Host health again after restarting...
checking host 10.33.2.98
ping host - ok
Waiting for ssh ready.
ssh - ok
\n
Wed, 17 Aug 2022 00:53:34 +0000 SSH to mds using 10.33.2.98 with  username
'aella'
```

Attach a virtual interface to Modular Sensor VM and connect it to the traffic monitoring bridge (br0-span)

1. Next, you need to attach a virtual interface to the Modular Sensor VM and connect it to the traffic monitoring bridge (br0-span in our example). Use the following command:

   ```
   virsh attach-interface --domain $hostname --type bridge --source $spanbr --
   model virtio --config --live --persistent
   ```

   Set the $hostname to the name you specified for your sensor VM during deployment (**mds** in this example).

2. Use the following command to verify that you can see the attached interface in the Modular Sensor VM. Substitute the name of your host if you did not use mds:

```
root@ubuntu-20-04-kvm:# virsh domiflist mds
Interface Type Source Model MAC
-------------------------------------------------------------
vnet0 bridge br0-aio virtio 52:54:00:f3:3a:63
vnet1 bridge br0-span virtio 52:54:00:38:11:80 <<< LOOK HERE
```

Disable TSO on Traffic Mirroring Interface

Modify the qemu script to disable TCP Segmentation Offload (TSO) on the traffic mirroring interface. You will need both the name of your VM (**mds** in our example) and the name of the interface (**ens192** in our example) to make these modifications. If you used other values, change the values in bold in the script below to match your own deployment.

1. Make a backup copy of the qemu script:

```
cd /etc/libvirt/hooks
cp qemu qemu.bak
```

2. Add the lines below to the script and save changes to the file:

```
#!/bin/bash
                                                    #
aella_hook_ds_span_mds_start
                                                    if [ "${1}" = "mds"
]; then
                                                    if [ "${2}" =
"start" ] || [ "${2}" = "reconnect" ]; then
                                                    i=0
                                                    str="ens192"
                                                    for port in $str
                                                    do
                                                    spanbr="br""${i}""-
span"
                                                    brctl setfd $spanbr
0
                                                    brctl setageing
$spanbr 0
                                                    ethtool -K $port
lro off
                                                    ethtool -K $port
gro off
                                                    i=`expr $i + 1`
                                                    done
                                                    fi
                                                    fi
                                                #
aella_hook_ds_span_mds_end
```

3. Stop the VM. For example:

   ```
   virsh shutdown mds
   ```

4. Wait for the `virsh list --all` command to show `shut off` for the VM.
5. Restart the VM:

   ```
   virsh start mds
   ```

Next Steps

Next, you'll apply a token to the sensor to authorize it and point it at the managing Stellar Cyber platform. These steps are the same for Ubuntu 20.04/22.04 as they are for Ubuntu 16.04/18.04. Proceed to **Applying a Token to the Installed Sensor**.

Installing in Ubuntu Server 16.04, 18.04, or CentOS 7.3

Click here to see the installation instructions for Ubuntu Server 16.04, 18.04, or CentOS 7.3:

Installation Steps: Bridge Mode

Install Sensor VM

You install the Modular Sensor VM by running the installation script downloaded from the DP. The syntax is different depending on whether you are installing the sensor with an IP address retrieved via DHCP or a static IP address. You use the --bridge parameter to specify the bridge for the management port and the --span parameter to specify which ports are monitored by the sensor.

Installing the Sensor with an Address from a DHCP Server

Use the following command to install the Modular Sensor with a management IP address retrieved from a DHCP server. Substitute the desired value in place of `mds1` for the hostname. Refer to **Available Parameters for the Installation Script** for a description of how each of these parameters are used.

```
sudo bash virt_deploy_modular_ds.sh -- --hostname=mds1 --span=eno2 --feature=mds --bridge=<bridgename>
```

Installing the Sensor with a Static IP Addresss

Use the following command to install the Modular Sensor with a static management IP address. Substitute the desired value in place of `mds1` for the hostname and use the appropriate IP configuration for your environment:

```
sudo bash virt_deploy_modular_ds.sh -- --hostname=mds1 --bridge=<bridgename> [--ip=192.168.1.223] [--netmask=<netmask>] [--gw=192.168.1.1] [--dns=8.8.8.8] [--dns-search=example.com] [--installdir=<imagedir>] [--span=eno2]  --feature=mds
```

Available Parameters for the Installation Script

You can supply the parameters listed and described below for the **virt_deploy_modular_ds.sh** script. Note that each parameter must start with the `--` string. This is required by the script.

- `--feature` determines the sensor type. If the value is `mds` the installation will be a modular sensor.
- `--hostname` specifies the name of the host. The VM name and the name of the sensor within Stellar Cyber are both set to this value.
- `--bridge` names the bridge to use for the management interface.
- `--ip` provides the static IPv4 address.
- `--netmask` must be set to the net mask of the form `255.255.255.0`.
- `--gateway` specifies the IP address of the gateway.
- `--dns` specifies the IP addresses of the DNS servers to use.
- `--dns-search` provides the default domain name for DNS searches.
- `--installdir` optionally specifies what directory will be used for the VM image installation.
- `--span` provides a list of host Ethernet ports to be included in the `aio-span` bridge.

When the script is executed it installs the VM, and creates a Linux bridge named `aio-span`. The ports in the `--span` parameter are added.

Installation Steps: NAT Mode

The process for installation in NAT mode is the same as for bridge mode except that the VM connects to the `virbr0` bridge that was created by the KVM installation.

The `ip` and associated parameters are set to a desired private address.

You must provide the necessary NAT services either through the host `iptables` system or externally to the box.

Applying a Token to the Installed Sensor

The next step in installing a Modular Sensor is to obtain and apply the token used to authorize and configure the installed sensor.

Obtaining a Token for the Installation

Tokens are required to authorize and configure the installation of a sensor image downloaded from the DP in the **System | DATA SOURCE MANAGEMENT | Sensors | Sensor Installation page**. Tokens point the installed sensor to the correct DP, assign the specified tenant, optionally provision a selected sensor profile, and authorize the sensor installation.

Use the following procedure to obtain a token in the Tokens tab:

1. Navigate to the **System | DATA SOURCE MANAGEMENT | Sensors | Sensor Installation** page and click on the **Tokens** tab.
2. If there is already an unexpired token that you want to use for the installation, you can either use the **Copy** button to copy it to the clipboard or use the **Download** button

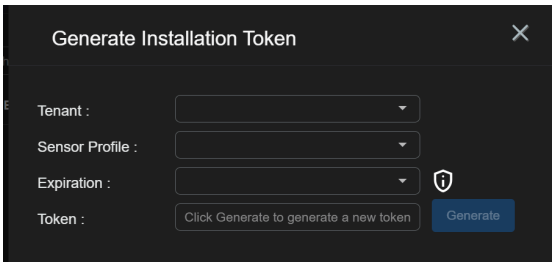
to download it as a file.

- Copy the token if you plan on applying it by pasting it into a `set token string <token>` command in the CLI.
- Download the token as a file if you plan on hosting the file on an HTTP server and referring to it in a `set token url <token url>` command.

Refer to **Assigning Tokens** for a summary of the different ways in which tokens can be applied to a sensor installation.

3. You can also click the **Generate** button to create a new token.

   The Generate Installation Token dialog appears:



4. Select the tenant for the token from the **Tenant** dropdown. This is the tenant to which all sensors authorized with this token will be automatically assigned. The dropdown lists all tenants configured for your organization in the **System | ORGANIZATION MANAGEMENT | Tenants** page.
5. Select the Sensor Profile to be assigned to all sensors authorized with this token from the **Sensor Profile** dropdown. The dropdown lists all sensor profiles available in the System | DATA SOURCE MANAGEMENT | Sensors | Sensor Profiles page for the selected Tenant.

   If you do not want to assign a Sensor Profile with a token, you can set this field to **None (no sensor profile selected)**. This is also the setting for any tokens migrated from a pre-5.3.0 release as part of an upgrade.

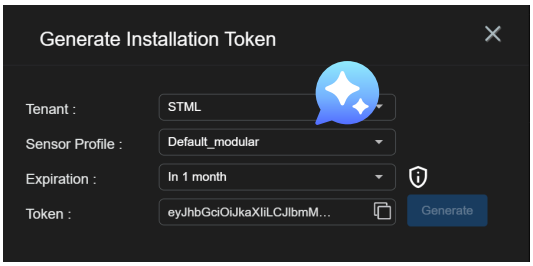
6. Use the **Expiration** dropdown to select an expiration date for this token. You can select specified expiration dates ranging from two weeks to three months.

   You can also select **Never expires** for the expiration date. However, Stellar Cyber recommends that you specify expiration dates for your tokens in order to improve the security of your deployment.

   Remember that when a token expires, **sensors authorized with the token continue to operate as normal**. Once a sensor successfully registers with the Stellar Cyber platform, it no longer uses the token. It is only used for the initial authorization, registration, and configuration of the sensor.

7. Click the **Generate** button.

   The system generates the token and displays its contents in the **Token** field. The dialog also updates to display the expiration date for the token, as illustrated below.



8. You can use the **Copy** button to copy the token to the clipboard immediately, or simply close the dialog and retrieve the token from the **Tokens** tab later on.

Applying the Token to the Sensor

Tokens are required to complete the installation of a sensor image downloaded from the DP in the **Download Image** tab.

You apply tokens to sensors as the last step in the overall installation procedure:

1. Log in to your new Sensor. The default username/password is **aella**/**changeme**. You are immediately prompted to change the password.
2. Apply the token to the installed sensor from the sensor CLI with the `set token` command using **one** of the options in the table below:



You only need to use **one** of the options in the table below. These are just two different ways to do the same thing – apply the token.

Option 1. Copy and Paste the Token String	Copy the token string from the **Tokens** tab and paste it into the CLI command. The syntax is as follows: `set token string <pasted string>`

Option 2. Host the Token on an HTTP Server	Download the token as a file from the **Tokens** tab, upload it to an HTTP server, and reference it in the `set token` command. The syntax is as follows:
	`set token url http://<url to token>`
	You can also use an HTTPS server. In that case, the specified URL must also include the username and password for the server using the following syntax:
	`set token url https://<user:password>@URL>`

3. The CLI reports that the **Sensor token is successfully set**.



If you receive an error message instead, it's possible that the token has expired. Refer to the **Tokens** tab to see the expiration date. If you are using the File technique, it's also possible that an extra space or line may have crept into your text file – check the file to make sure it includes only the token text.

4. Wait a minute or so. Then, verify that the token was successfully applied using any combination of the following techniques:
    - Check the **System | DATA SOURCE MANAGEMENT | Sensors | Sensors** list in the user interface to see that the sensor has registered itself successfully.
    - Verify that the `show system` command shows all services as running.

- Verify that the `show receiver` command displays a receiver.
- Verify that the `show json` command reports some data sent in the `BYTE_SENT` column.

Configuring a Static IP Address (Optional)

By default, the sensor uses DHCP for the management port's IP address. For ease of troubleshooting, however, Stellar Cyber recommends that you reconfigure the management port to use a static IP address. The procedure is as follows:

1. Log in to your sensor. The default username/password is **aella**/**changeme**, but you changed this when you applied the token in the previous section.
2. You can set IP parameters manually using commands similar to the following (substitute your own IP parameters for the ones shown in bold below):

   ```
   set interface management ip 192.168.14.100/255.255.255.0
   set interface management gateway 192.168.14.1
   set interface management dns 8.8.8.8
   ```

3. Verify the IP settings with the `show interfaces` command.
4. Log out with the `quit` command.

Configure NTP and Set the Timezone

Stellar Cyber strongly recommends that configure NTP and set the timezone for the sensor.

Refer to **Best Practices for NTP and Timezones** for details.

Enabling SSSE3 for the Modular Sensor VM

The Modular Sensor VM must have SSSE3 enabled for its processors in order to operate correctly. In most cases, SSSE3 will already be enabled. However, if you encounter issues with packet collection or Interflow data generation, you can use the instructions below to ensure that SSSE3 is enabled.

Click here to see instruction on how to enable SSSE3.



Sensors installed on Linux hosts must have SSSE3 enabled for their processors in order to operate correctly. This is true for Modular Sensors and Linux Server Sensors, as well as legacy Network and Security Sensors.
SSSE3 is typically supported/enabled for most vCPUs, but may not be for certain legacy AMD vCPUs. See below for instructions on enabling SSSE3.

To enable SSSE3 for a virtual machine:

1. Start the virtual shell (virsh)
2. Type the following command to edit the virtual machine's settings:

   ```
   edit <virtual_machine_name>
   ```

3. Locate the **<cpu>** section. It should appear similar to the following:

```
<cpu mode='custom' match='exact' check='partial'>
    <model fallback='allow'>SandyBridge</model>
......
 </cpu>
```

4. Add the following line to the **<cpu>** section:

   ```
   <feature policy='require' name='ssse3'/>
   ```

   When you are done, the **<cpu>** section should appear similar to the following:

```
<cpu mode='custom' match='exact' check='partial'>
    <model fallback='allow'>SandyBridge</model>
    <feature policy='require' name='ssse3'/>
...........
  </cpu>
```

5. Save changes to the virtual machine and exist virsh.
6. Run the following command:

   ```
   virsh define /etc/libvirt/qemu/<virtual_machine_name>.xml
   ```

   Each virtual machine has a configuration .xml file. Typically, these files are stored under **/etc/libvirt/qemu**, but the location may be different for your system.

7. Stop and start the virtual machine in virsh.

© 2026 Stellar Cyber . All rights reserved.